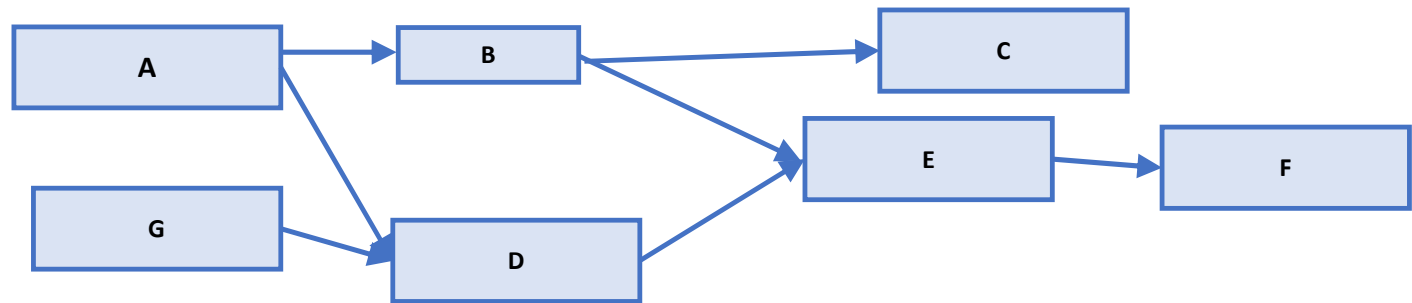


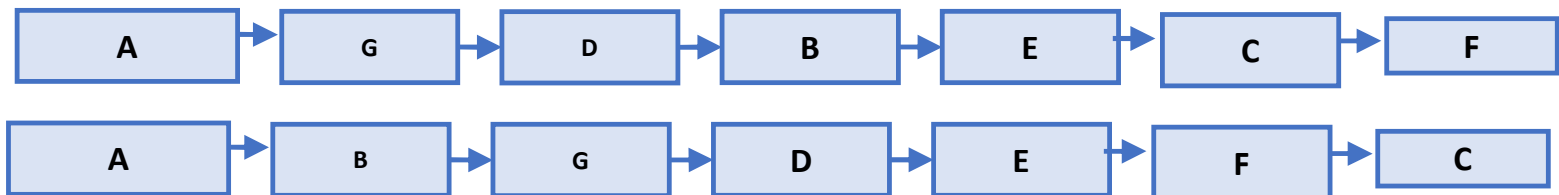
Topological Sort

Sorting Dependencies

- Given a set of courses and their prerequisites, find an order to take the courses in (assuming you can only take one course per quarter)



- Possible ordering:



2021-23 Catalog Requirements



A
G.E. Oral
Communication

A
G.E. Freshman
Composition

Denotes major requirement that also satisfies G.E. Section A, B or E.

Placement in courses leading to major requirements is dependent on high school preparation, MPT scores and current computer programming skills. Consult with an academic advisor for proper placement.

(#) MATH 482 has as one of its prerequisites MATH 262; COMP 482 does not have MATH 262 as one of its prerequisites.

(##) Check senior electives for their prerequisites, many require just COMP 380/L, some require just COMP 322/L, some require both COMP 322/L and COMP 380/L, and a few require COMP 310.

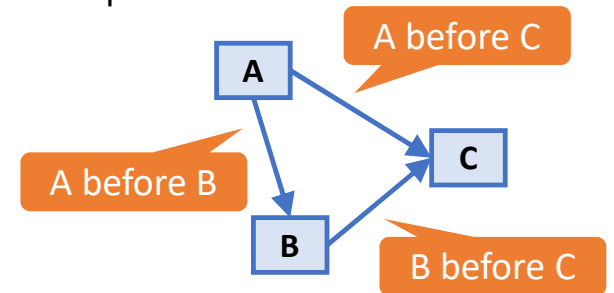
A Graduation Evaluation is required 2 to 3 semesters prior to planned date of graduation.

Always check the online University Catalog for the latest requirements of the major.

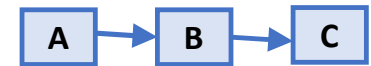
Topological Sort

- A **topological sort** of a directed graph G without cycles is an ordering of the nodes, where for every edge in the graph, the origin appears before the destination in the ordering
- Intuition: a “dependency graph”
 - An edge (u, v) means u must happen before v
 - A topological sort of a dependency graph gives an ordering that **respects dependencies**
- Applications:
 - Graduating
 - Compiling multiple Java files
 - Multi-job Workflows

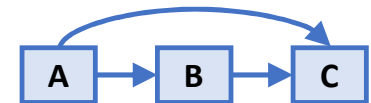
Input:



Topological Sort:

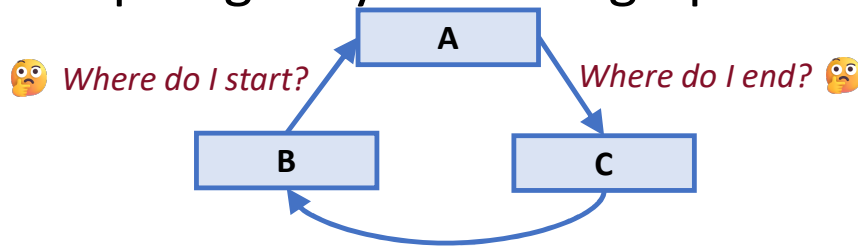


With original edges for reference:



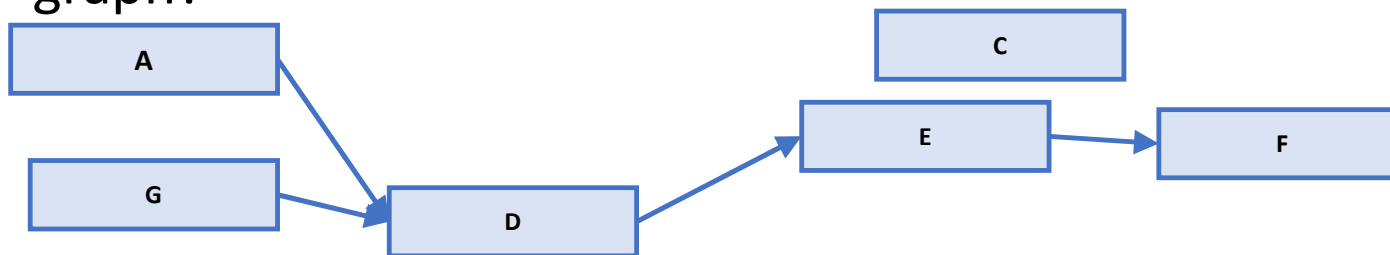
Can We Always Topo Sort a Graph?

- Can you topologically sort this graph?



No 🤔

- What's the difference between this graph and our first graph?



- A graph has a topological ordering iff it is a DAG
 - But a DAG can have multiple orderings

DIRECTED ACYCLIC GRAPH

- A **directed graph** without any **cycles**
- Edges may or may not be weighted

How To Perform Topo Sort?

- Topo sort is an ordering problem. Could we use... BFS?

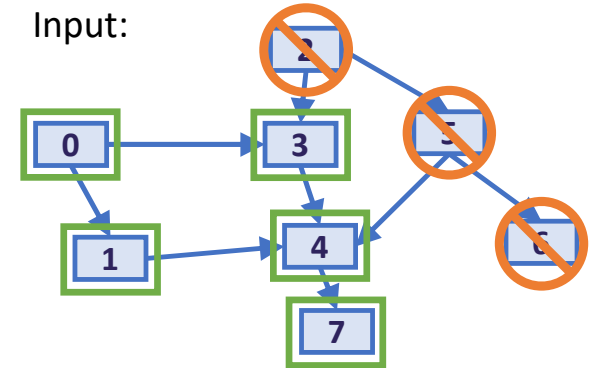
IDEA 1

Performing Topo Sort

Use BFS, starting from a vertex with no incoming edges

Doesn't reach all vertices ☹️

Input:



BFS starting at 0:



How To Perform Topo Sort?

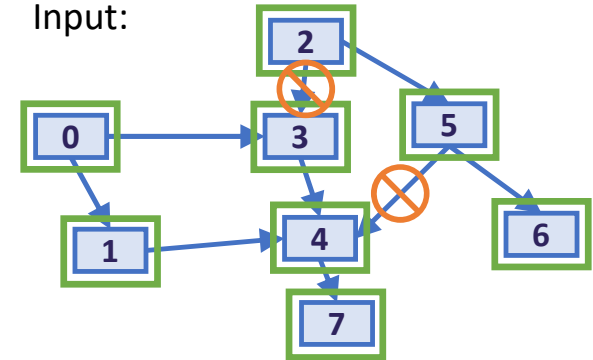
- Okay, there may be multiple “roots”. What if we use BFS multiple times?

IDEA 2

Performing Topo Sort

Use BFS, starting from ALL vertices with no incoming edges

Input:



BFS starting at 0:

+ BFS starting at 2:



How To Perform Topo Sort?

- Okay, there may be multiple “roots”. What if we use BFS multiple times?

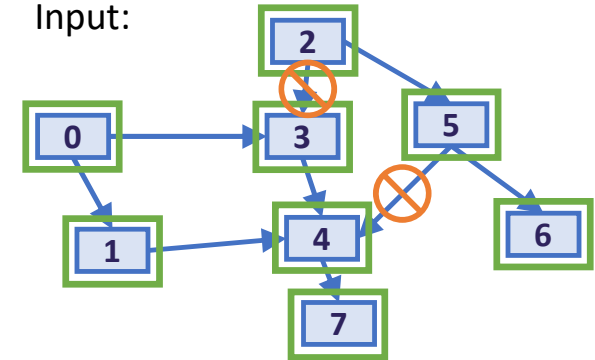
IDEA 2

Performing Topo Sort

Use BFS, starting from ALL vertices with no incoming edges

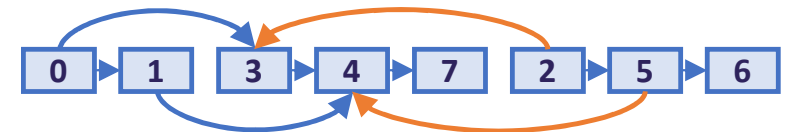
Doesn't respect all edges ☹️

Input:



BFS starting at 0:

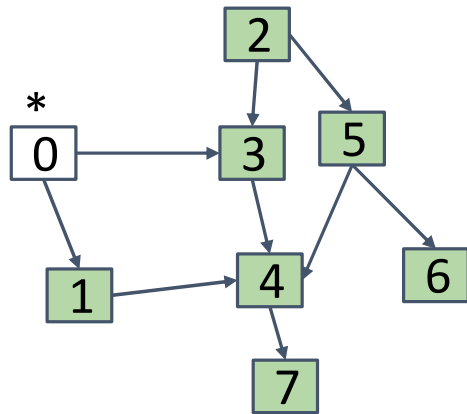
+ BFS starting at 2:



Idea: DFS:

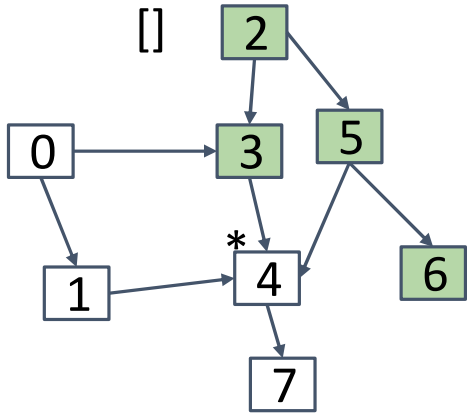
- In DFS, we print a vertex and then recursively call DFS for its adjacent vertices. In topological sorting, we need to print a vertex before its adjacent vertices.
- We can modify DFS to find the Topological Sorting of a graph. In DFS,
 - We start from a vertex, we first print it, and then
 - Recursively call DFS for its adjacent vertices.
- In topological sorting,
 - We use a temporary stack.
 - We don't print the vertex immediately,
 - We first recursively call topological sorting for all its adjacent vertices, then push it to a stack.
 - Finally, print the contents of the stack.
- [Demo for topSort](#)

Topological Sort (Demo 1/2)

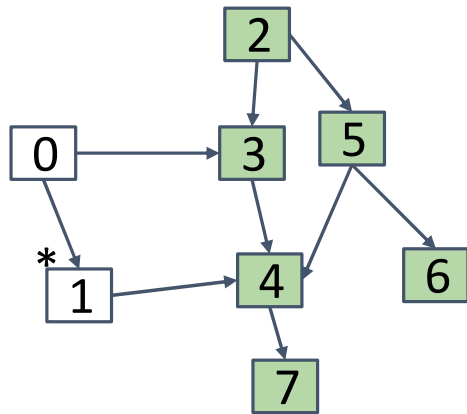


Postorder:

[]

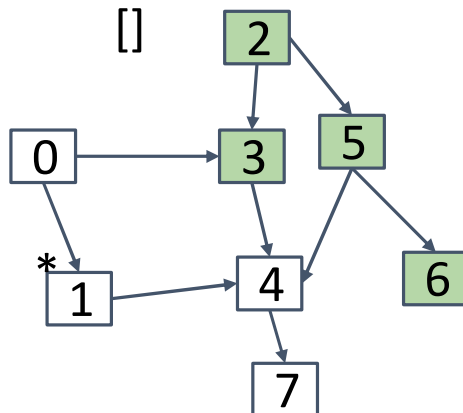


Postorder: [7, 4]

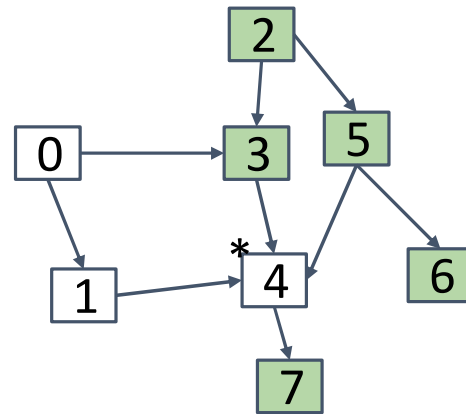


Postorder:

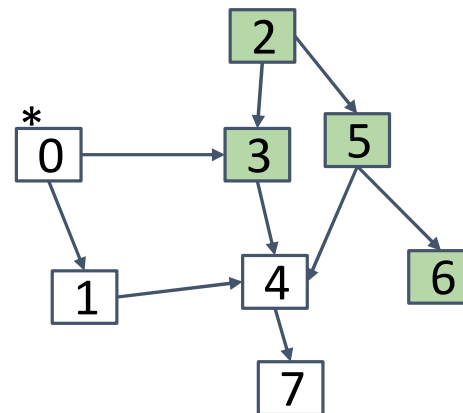
[]



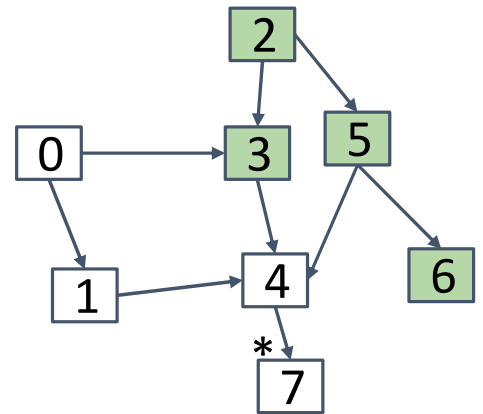
Postorder: [7, 4, 1]



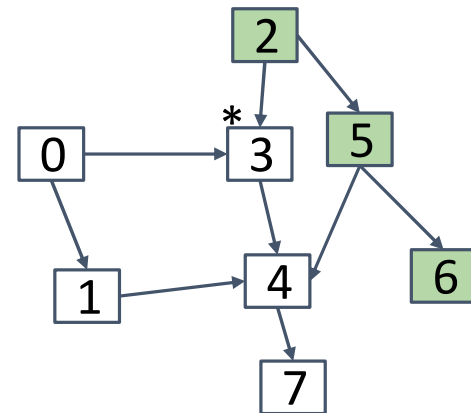
Postorder: []



Postorder: [7, 4, 1]

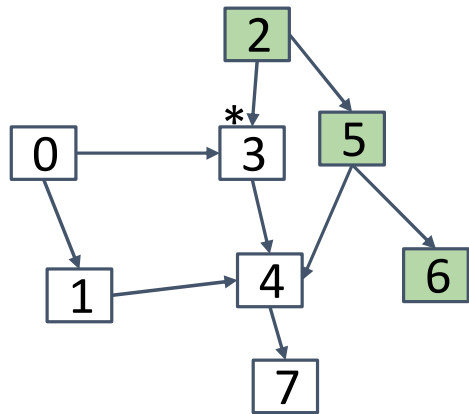


Postorder: [7]

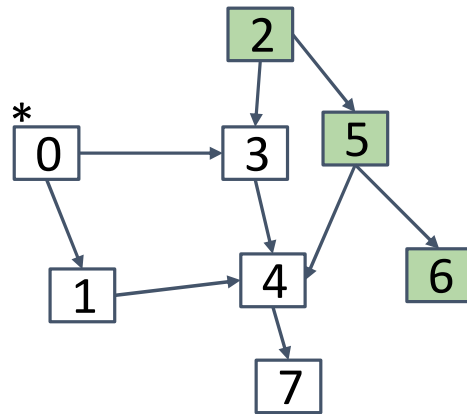


Postorder: [7, 4, 1, 3]

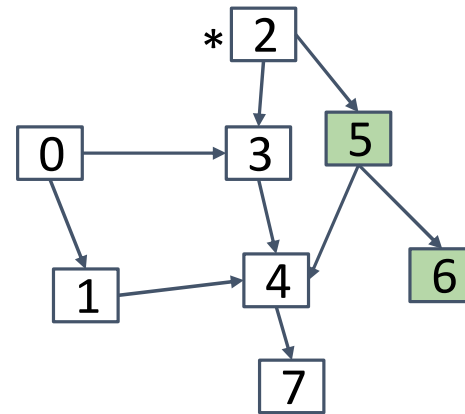
Topological Sort (Demo 2/2)



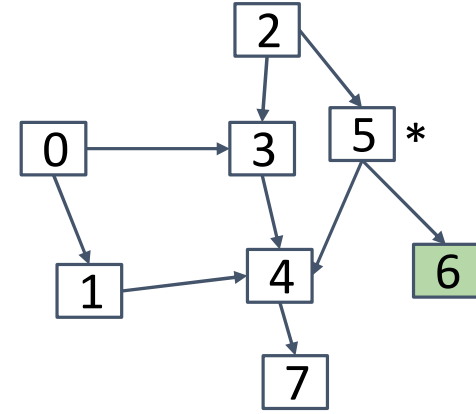
Postorder: [7, 4, 1, 3]



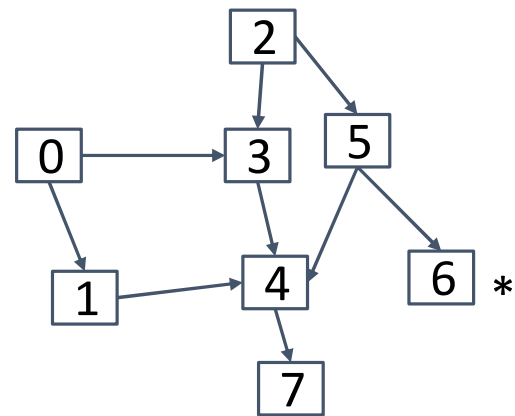
Postorder: [7, 4, 1, 3, 0]



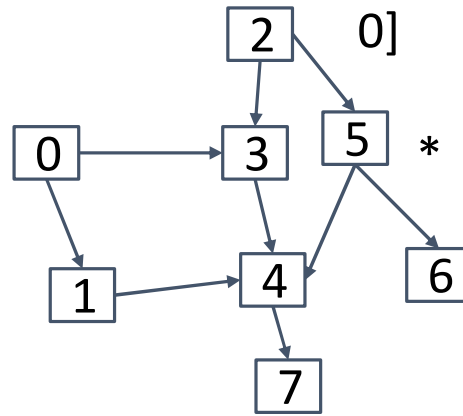
Postorder: [7, 4, 1, 3, 0]



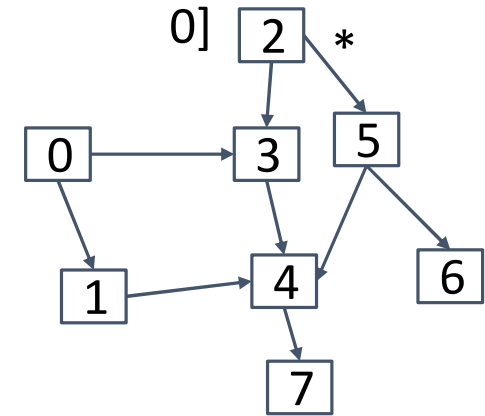
Postorder: [7, 4, 1, 3, 0]



Postorder: [7, 4, 1, 3, 0, 6, 5]

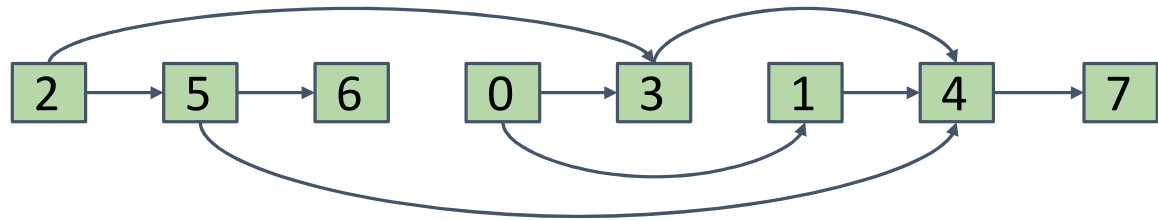
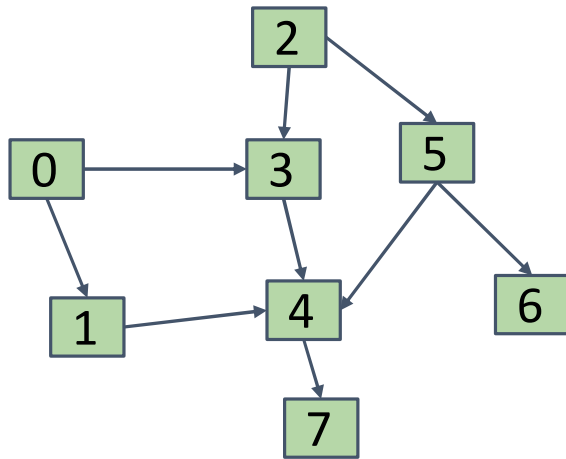


Postorder: [7, 4, 1, 3, 0, 6, 5]



Postorder: [7, 4, 1, 3, 0, 6, 5]

Topological Sort



The reason it's called topological sort: Can think of this process as sorting our nodes so they appear in an order consistent with edges, e.g. [2, 5, 6, 0, 3, 1, 4, 7]

- When nodes are sorted in diagram, arrows all point rightwards.

None Recursive Algorithm

1. Created visited list
2. While the visited list does not contain all the vertices
 - Find a vertex that has no successor
 - Remove all the edges that lead to it from edgeList
 - Add it to the visited list
 - Add it to the topological list